

legal-retrieval-benchmark: hybrid retrieval over legal corpora

Akshitha Reddy Lingampally

2026-06-06

Contents

1	Abstract	2
2	1. Background	2
2.1	1.1 Motivation	3
2.2	1.2 Scope	3
3	2. Related Work	3
4	3. Method	3
4.1	3.1 Architecture	3
4.2	3.2 BM25	4
4.3	3.3 Dense retrieval	4
4.4	3.4 RRF hybrid	4
4.5	3.5 Cross-encoder reranking	4
5	4. Data	4
5.1	4.1 CUAD-QA (primary)	4
5.2	4.2 LegalBench-RAG (future)	5
5.3	4.3 In-repo fixture	5
6	5. Evaluation Setup	5
6.1	5.1 Metrics	5
6.2	5.2 Hardware	5
6.3	5.3 Re-run cost	5
7	6. Results	5
8	7. Ablations	6
8.1	7.1 RRF over-fetch sweep	6
8.2	7.2 Length normalization in BM25	6
9	8. Discussion	6
10	9. Limitations	7
11	10. Future Work	7

12 11. References	7
13 Appendix A. Reproducibility Checklist	8

1 Abstract

We present legal-retrieval-benchmark, a reproducible harness for comparing four retrieval recipes on legal corpora: BM25, dense bi-encoder retrieval, RRF hybrid fusion of the two, and cross-encoder reranking. We evaluate each recipe on a 1,000-query, 36-contract slice of CUAD (Hendrycks et al., 2021), demonstrating that BM25 ($\text{nDCG}@10 = 0.245$) substantially outperforms the BGE-small dense retriever ($\text{nDCG}@10 = 0.133$) and the RRF fusion of the two ($\text{nDCG}@10 = 0.230$) on this corpus. We trace the dense underperformance to the 512-token context window of BGE-small running against 30K-70K character contracts, and identify chunked indexing as the obvious remediation. The harness ships with the LegalBench-RAG span-level evaluation as planned future work and a clean Python interface so any new retriever can be added in under 50 lines.

2 1. Background

Retrieval-Augmented Generation has become the default architecture for legal-domain language model applications, but the retriever component is often treated as a solved problem and instantiated with whichever vector database is at hand. This is a mistake for two reasons. First, dense bi-encoder retrievers have a documented weak spot on long structured documents like contracts and judicial opinions: the encoder’s context window forces truncation, and the truncated tail often contains the relevant clause. Second, the cost of running production-scale ANN indexing on 100K+ legal documents materially exceeds the cost of a classical BM25 baseline, and the quality tradeoff is rarely measured directly.

This project answers a narrow but practically important question: for a given legal corpus, which retrieval recipe is actually best, and how much do the extra stages cost? We compare BM25 (Robertson & Walker, 1994), a modern dense bi-encoder (BGE-small-en-v1.5; Xiao et al., 2024), reciprocal rank fusion (Cormack et al., 2009), and cross-encoder reranking (MiniLM L-6). Each retriever exposes the same Index interface and runs through a common evaluation harness that records build time, search time (QPS), and the standard IR metrics ($\text{nDCG}@k$, $\text{Recall}@k$, $\text{MRR}@k$, $\text{MAP}@k$).

The corpus we ship as the default benchmark is the SQuAD-style CUAD dataset (Hendrycks et al., 2021): 510 commercial contracts and 22,450 clause-type questions. For the first published results in this report we sub-sample to 1,000 queries $\times$ 36 contracts so the harness fits on a CPU laptop. LegalBench-RAG (Pipitone & Alami, 2024) ships span-level annotations for four sub-corpora (contractnli, cuad, maud, privacy_qa); span-level evaluation is the next major milestone.

2.1 1.1 Motivation

A representative production legal RAG system today reaches for a hosted vector database, ingests a corpus once, and never re-evaluates whether the default recipe is the right one for the data. This is operationally convenient but technically lazy: the quality difference between BM25 and dense retrieval on a particular corpus is often 10-20 nDCG points and goes in unpredictable directions depending on the corpus characteristics. The goal of this project is to make that comparison cheap enough that nobody has an excuse to skip it.

2.2 1.2 Scope

This report covers the harness design, the dataset preparation pipeline, the four retrieval recipes, the evaluation protocol, and our first real-numbers run on CUAD. Section 11 lists the references that informed the design. Future work is in Section 10.

3 2. Related Work

Lexical retrieval. BM25 (Robertson & Walker, 1994) remains the dominant classical baseline. We use the rank-bm25 implementation with $k_1=1.5$ and $b=0.75$, matching the values used in the original BEIR benchmarks (Thakur et al., 2021).

Dense bi-encoders. The BGE family (Xiao et al., 2024) is currently the strongest open-weight English bi-encoder under 200M parameters. We use BGE-small-en-v1.5 (33M parameters, 384-dim embeddings) because it runs on CPU; production deployments would substitute BGE-large or a legal-domain model when one becomes available with a clear license.

Fusion methods. Reciprocal Rank Fusion (Cormack et al., 2009) is the go-to method when score normalization across heterogeneous rankers is fragile. We use $k=60$ (the value the original paper found robust across TREC tasks) and over-fetch by 4x before fusion.

Cross-encoder reranking. The two-stage retriever pattern (cheap recall stage followed by expensive precision stage) is standard. We use cross-encoder/ms-marco-MiniLM-L-6-v2 because it is small enough to rerank the top-50 of any base on CPU in under 200ms per query.

Legal RAG. LegalBench-RAG (Pipitone & Alami, 2024) is the closest direct comparison; we follow their corpus + queries + qrels layout but do not yet run their span-level evaluation (see Section 10).

4 3. Method

4.1 3.1 Architecture

The harness has four primary modules under `src/legal_rag_benchmark/`: `data/loader.py` for CUAD-QA and JSONL loaders, `retrievers/` for the four Retriever implementations, `eval/metrics.py` for the IR metric math, and `eval/runner.py` for orchestration. Each

Retriever implements `index(documents) -> None` and `search(query, top_k) -> list[Hit]`. The runner times the two phases separately so we can report build cost and search cost as independent columns. All metrics are macro-averaged across queries; per-query scores are saved as JSONL for downstream analysis.

4.2 3.2 BM25

We tokenize on `\w+`, lowercase, and drop tokens of length 1. The title field (when present) is concatenated to the body. We use rank-bm25's `BM25Okapi` with `k1=1.5` and `b=0.75`.

4.3 3.3 Dense retrieval

We encode each document with BGE-small-en-v1.5 in batches of 32, with L2 normalization so that the resulting FAISS `IndexFlatIP` inner-product score equals cosine similarity. The max sequence length is 512 tokens, which truncates documents longer than that to their prefix.

4.4 3.4 RRF hybrid

For each query, we over-fetch `base_k = max(top_k × 4, 50)` results from each base retriever. We then compute the RRF score $\text{score}(d) = \sum \text{over } r \text{ of } 1/(k + \text{rank}_r(d))$ with `k=60`. Documents that don't appear in a retriever's top `base_k` get zero contribution from that retriever.

4.5 3.5 Cross-encoder reranking

The reranker wraps any base retriever. It pulls the top-50 from the base, runs (query, doc) pairs through the cross-encoder in batches of 32, and returns the top-k by cross-encoder score. The base retriever's score is discarded after the rerank.

5 4. Data

5.1 4.1 CUAD-QA (primary)

The Contract Understanding Atticus Dataset (CUAD; Hendrycks et al., 2021) contains 510 commercial contracts with 41 expert-labeled clause types per contract. The HuggingFace mirror `theatticusproject/cuad-qa` exposes it as SQuAD-style (context, question, answers) rows. We collapse to: one document per unique contract title, one query per non-empty SQuAD row, and `qrels {contract_title: 1}` for the contract the answer came from. A single CUAD question is templated (the same 41 questions repeat across all contracts), so we suffix the `qid` with the contract title to keep one-relevant-doc-per-query. For the first reported run we sub-sample to 1,000 queries × 36 unique contracts.

5.2 4.2 LegalBench-RAG (future)

LegalBench-RAG (Pipitone & Alami, 2024) ships four sub-corpora with span-level annotations. The source documents themselves live in the project’s GitHub release at `github.com/zeroentropy-ai/legalbenchrag` and are not on HuggingFace; wiring the fetcher and switching to span-level qrels is Section 10.

5.3 4.3 In-repo fixture

A tiny 8-document, 4-query fixture is checked into `tests/fixtures/` so the CI run does not touch the network. The fixture is hand-written and exercises every code path; full benchmarks run from the prepared data directory.

6 5. Evaluation Setup

6.1 5.1 Metrics

- **nDCG@k**: normalized discounted cumulative gain. We use binary relevance (CUAD gives only present/absent per clause).
- **Recall@k**: fraction of relevant documents found in the top-k.
- **MRR@k**: reciprocal rank of the first relevant document.
- **MAP@k**: mean average precision over the top-k.
- **QPS**: queries per second, end-to-end including any in-process reranking. Wall-clock, single-threaded Python.

6.2 5.2 Hardware

Apple M-series CPU, no GPU. The dense and cross-encoder models load to MPS when available.

6.3 5.3 Re-run cost

A single full run (4 retriever variants × 1K queries × 36-document corpus) takes under 5 minutes end-to-end on the reference hardware. The dense index build dominates (~60% of total time); BM25 is ~0.3% of total.

7 6. Results

The headline table:

retriever	nDCG@10	Recall@10	MRR@10	MAP@10	QPS
bm25	0.245	0.493	0.171	0.171	4948
dense (BGE-small)	0.133	0.301	0.084	0.084	135

retriever	nDCG@10	Recall@10	MRR@10	MAP@10	QPS
rfr(bm25+dense)	0.230	0.459	0.161	0.161	159

Three things worth being explicit about:

1. **BM25 wins on CUAD by a wide margin.** This contradicts the textbook “dense beats sparse” expectation. The cause is structural: the CUAD contracts run 30K-70K characters each, and BGE-small with a 512-token cap only sees the first ~2,000 characters. The tail of the contract — which often contains the actual clause being asked about — is invisible to the dense encoder. BM25 indexes the whole document.
2. **RRF does not rescue the dense ranker.** RRF gives equal rank-based weight to both base retrievers. When one base is much weaker than the other (as dense is here), the fusion is dragged below the stronger ranker. This is exactly the failure mode the RRF authors warn about in Cormack et al. (2009); we reproduce it cleanly here.
3. **The fix is chunked dense indexing.** Splitting each contract into ~512-token chunks, indexing every chunk, and aggregating chunk scores back to document level (max or sum) is the canonical solution. The chunk-aware dense ranker should overtake BM25 on this corpus.

8 7. Ablations

This iteration ships one substantive ablation (the RRF base_k sweep) and one negative result (length-aware BM25 weighting).

8.1 7.1 RRF over-fetch sweep

We swept over-fetch $\in \{2, 4, 8, 16\}$. nDCG@10 is monotonically non-decreasing with over-fetch but the gain saturates at over-fetch=4. We use 4 as the default in the runner.

8.2 7.2 Length normalization in BM25

We tried $b=0.0$ (no length normalization) and $b=1.0$ (full length normalization). Both performed worse than the default $b=0.75$. The CUAD corpus has high variance in document length, so partial normalization is the sensible default.

9 8. Discussion

The most important finding is not which retriever wins; it is that the answer is corpus-dependent and goes in the non-intuitive direction here. A team that picked dense retrieval on the basis of “this is what modern RAG systems use” would have shipped a system that retrieves the wrong contract

more than half the time on CUAD-style clause questions. The harness makes that comparison cheap; the lesson is that the comparison needs to be run, not assumed.

A secondary finding: build cost matters when the corpus changes frequently. BM25 builds in 0.28 seconds on this corpus; dense takes 66 seconds. For a corpus that gets a fresh batch of contracts daily, the build cost difference adds up.

10 9. Limitations

1. **Doc-level qrels only.** CUAD’s answers are spans, but we collapse to “the contract that contains the answer.” Span-level eval is the LegalBench-RAG protocol and is queued.
2. **Subsample.** 1,000 queries × 36 contracts is small. The full 22K queries × 510 contracts is what production deployments need.
3. **Single encoder.** Only BGE-small was evaluated on the dense side. Legal-BERT or SaulLM embeddings might close the gap.
4. **No chunked dense.** The dense underperformance is mechanistic (truncation). The chunked variant is the obvious next step.
5. **Single hardware target.** CPU-only; GPU profile would differ but the rank order should be stable.

11 10. Future Work

- ☐ Chunked dense indexing (the headline-changing item).
- ☐ LegalBench-RAG span-level evaluation.
- ☐ ColBERT-style late interaction as a fourth base retriever.
- ☐ Legal-domain embedders once licenses are clear.
- ☐ Query-side prompting (HyDE) to lift dense recall.
- ☐ Per-query cost-aware routing.

12 11. References

- Cormack, G. V., Clarke, C. L. A., & Buettcher, S. (2009). *Reciprocal Rank Fusion outperforms Condorcet and individual rank learning methods*. SIGIR.
- Hendrycks, D., Burns, C., Chen, A., & Ball, S. (2021). *CUAD: An Expert-Annotated NLP Dataset for Legal Contract Review*. NeurIPS Datasets and Benchmarks.
- Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press.
- Pipitone, N., & Alami, G. (2024). *LegalBench-RAG: A Benchmark for Retrieval-Augmented Generation in the Legal Domain*. arXiv:2408.10343.
- Robertson, S. E., & Walker, S. (1994). *Some simple effective approximations to the 2-Poisson model for probabilistic weighted retrieval*. SIGIR.

- Thakur, N., Reimers, N., Rüklé, A., Srivastava, A., & Gurevych, I. (2021). *BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models*. NeurIPS.
- Xiao, S., Liu, Z., Zhang, P., Muennighoff, N., Lian, D., & Nie, J.-Y. (2024). *C-Pack: Packed Resources For General Chinese Embeddings*. SIGIR.

13 Appendix A. Reproducibility Checklist

- ☒ Code open-source under MIT.
- ☒ All hyperparameters surfaced through CLI + pyproject.toml defaults.
- ☒ All random seeds fixed in the runner.
- ☒ All datasets downloaded from public sources (HuggingFace theatticusproject/cuad-qa).
- ☒ Test artifacts in docs/test_results/.
- ☒ Per-query results in results/cuad__<retriever>__runs.jsonl.
- ☒ Aggregated metrics in results/cuad__<retriever>__metrics.json.